

Stat 184 4.3 QMD Andrew McRavey

Andrew McRavey

Part 1: Busiest Airports Analysis

1.1 Tidying/Table Creation Function:

```
library(tidyverse)
```

Warning: package 'tidyverse' was built under R version 4.5.3

Warning: package 'stringr' was built under R version 4.5.3

Warning: package 'lubridate' was built under R version 4.5.3

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.0      v readr      2.1.6
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.2      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr      1.2.1
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
airports_wide <- tribble(
  ~iata_code, ~airport_name, ~city, ~country, ~region,
  ~`2020`, ~`2021`, ~`2022`, ~`2023`, ~`2024`, ~`2025`,
  "ATL", "Hartsfield-Jackson Atlanta Intl", "Atlanta", "United States", "North America")
```

```

42.92, 75.70, 93.70, 104.65, 108.07, 106.30,
"FRA", "Frankfurt Airport", "Frankfurt", "Germany", "Europe",
18.77, 24.81, 48.92, 59.36, 61.56, 63.19,
"PKX", "Beijing Daxing Intl", "Beijing", "China", "East Asia",
16.00, 25.00, 10.23, 39.41, 49.42, 56.00,
"DXB", "Dubai International", "Dubai", "UAE", "Middle East",
25.84, 29.11, 66.10, 86.90, 92.30, 95.20,
"LHR", "London Heathrow", "London", "United Kingdom", "Europe",
22.11, 19.39, 61.60, 79.20, 83.86, 84.46,
"DFW", "Dallas/Fort Worth Intl", "Dallas-Fort Worth", "United States", "North America",
39.40, 62.50, 73.40, 81.80, 87.80, 90.00
)

# 2. Pivot to tidy long format
# One row = one airport × one year

airports_tidy <- airports_wide |>
  pivot_longer(
    cols = `2020`:`2025`,
    names_to = "year",
    values_to = "passengers_millions"
  ) |>
  mutate(year = as.integer(year)) |>
  arrange(iata_code, year)

# 3. Add year-over-year change columns (within each airport)

airports_tidy <- airports_tidy |>
  group_by(iata_code) |>
  mutate(
    yoy_change_millions = passengers_millions - lag(passengers_millions),
    yoy_pct_change = round((passengers_millions / lag(passengers_millions) - 1) * 100, 1)
  ) |>
  ungroup()

```

```
# 4. Enforce factor ordering for airports (by 2024 traffic)

airport_order <- airports_tidy |>
  filter(year == 2024) |>
  arrange(desc(passengers_millions)) |>
  pull(iata_code)

airports_tidy <- airports_tidy |>
  mutate(iata_code = factor(iata_code, levels = airport_order))

# 5. Preview

glimpse(airports_tidy)
```

```
Rows: 36
Columns: 9
$ iata_code      <fct> ATL, ATL, ATL, ATL, ATL, ATL, DFW, DFW, DFW, DFW, ~
$ airport_name   <chr> "Hartsfield-Jackson Atlanta Intl", "Hartsfield-
Jac~
$ city           <chr> "Atlanta", "Atlanta", "Atlanta", "Atlanta", "Atlan~
$ country        <chr> "United States", "United States", "United States",~
$ region         <chr> "North America", "North America", "North America",~
$ year           <int> 2020, 2021, 2022, 2023, 2024, 2025, 2020, 2021, 20~
$ passengers_millions <dbl> 42.92, 75.70, 93.70, 104.65, 108.07, 106.30, 39.40~
$ yoy_change_millions <dbl> NA, 32.78, 18.00, 10.95, 3.42, -1.77, NA, 23.10, 1~
$ yoy_pct_change  <dbl> NA, 76.4, 23.8, 11.7, 3.3, -1.6, NA, 58.6, 17.4, 1~
```

```
print(airports_tidy, n = 36)
```

```
# A tibble: 36 x 9
  iata_code airport_name      city country region  year passengers_millions
  <fct>      <chr>          <chr> <chr>   <chr> <int>         <dbl>
1 ATL      Hartsfield-Jackson ~ Atla~ United~ North~  2020         42.9
2 ATL      Hartsfield-Jackson ~ Atla~ United~ North~  2021         75.7
3 ATL      Hartsfield-Jackson ~ Atla~ United~ North~  2022         93.7
4 ATL      Hartsfield-Jackson ~ Atla~ United~ North~  2023        105.
5 ATL      Hartsfield-Jackson ~ Atla~ United~ North~  2024        108.
6 ATL      Hartsfield-Jackson ~ Atla~ United~ North~  2025        106.
7 DFW      Dallas/Fort Worth I~ Dall~ United~ North~  2020         39.4
8 DFW      Dallas/Fort Worth I~ Dall~ United~ North~  2021         62.5
```

9	DFW	Dallas/Fort Worth I~	Dall~	United~	North~	2022	73.4
10	DFW	Dallas/Fort Worth I~	Dall~	United~	North~	2023	81.8
11	DFW	Dallas/Fort Worth I~	Dall~	United~	North~	2024	87.8
12	DFW	Dallas/Fort Worth I~	Dall~	United~	North~	2025	90
13	DXB	Dubai International	Dubai	UAE	Middl~	2020	25.8
14	DXB	Dubai International	Dubai	UAE	Middl~	2021	29.1
15	DXB	Dubai International	Dubai	UAE	Middl~	2022	66.1
16	DXB	Dubai International	Dubai	UAE	Middl~	2023	86.9
17	DXB	Dubai International	Dubai	UAE	Middl~	2024	92.3
18	DXB	Dubai International	Dubai	UAE	Middl~	2025	95.2
19	FRA	Frankfurt Airport	Fran~	Germany	Europe	2020	18.8
20	FRA	Frankfurt Airport	Fran~	Germany	Europe	2021	24.8
21	FRA	Frankfurt Airport	Fran~	Germany	Europe	2022	48.9
22	FRA	Frankfurt Airport	Fran~	Germany	Europe	2023	59.4
23	FRA	Frankfurt Airport	Fran~	Germany	Europe	2024	61.6
24	FRA	Frankfurt Airport	Fran~	Germany	Europe	2025	63.2
25	LHR	London Heathrow	Lond~	United~	Europe	2020	22.1
26	LHR	London Heathrow	Lond~	United~	Europe	2021	19.4
27	LHR	London Heathrow	Lond~	United~	Europe	2022	61.6
28	LHR	London Heathrow	Lond~	United~	Europe	2023	79.2
29	LHR	London Heathrow	Lond~	United~	Europe	2024	83.9
30	LHR	London Heathrow	Lond~	United~	Europe	2025	84.5
31	PKX	Beijing Daxing Intl	Beij~	China	East ~	2020	16
32	PKX	Beijing Daxing Intl	Beij~	China	East ~	2021	25
33	PKX	Beijing Daxing Intl	Beij~	China	East ~	2022	10.2
34	PKX	Beijing Daxing Intl	Beij~	China	East ~	2023	39.4
35	PKX	Beijing Daxing Intl	Beij~	China	East ~	2024	49.4
36	PKX	Beijing Daxing Intl	Beij~	China	East ~	2025	56

i 2 more variables: yoy_change_millions <dbl>, yoy_pct_change <dbl>

1.2 Plotting Function:

```
library(tidyverse)

# Plot-specific wrangling: label data for direct line labels

label_data <- airports_tidy |>
  filter(year == 2025)

# Plot
```

```

ggplot(airports_tidy, aes(x = year, y = passengers_millions,
                        color = iata_code, group = iata_code)) +

# COVID dip shading
annotate("rect",
        xmin = 2019.5, xmax = 2021.5,
        ymin = -Inf, ymax = Inf,
        fill = "grey90", alpha = 0.5) +
annotate("text",
        x = 2020.5, y = 108,
        label = "COVID-19\nrestrictions",
        size = 3, color = "grey50", fontface = "italic") +

# Lines and points
geom_line(linewidth = 1.1) +
geom_point(size = 2.5) +

# Direct endpoint labels (replaces legend)
geom_text(data = label_data,
        aes(label = paste0(iata_code, "\n", passengers_millions, "M")),
        hjust = -0.1,
        size = 3.2,
        fontface = "bold",
        show.legend = FALSE) +

# Axis formatting
scale_x_continuous(
    breaks = 2020:2025,
    expand = expansion(mult = c(0.03, 0.16)) # right margin for labels
) +
scale_y_continuous(
    breaks = seq(0, 120, by = 20),
    labels = scales::label_number(suffix = "M"),
    limits = c(0, 120)
) +
scale_color_brewer(palette = "Dark2") +

# Labels
labs(
    title = "Annual Passenger Traffic at Six Major Airports (2020-2025)",
    subtitle = "Total passengers (enplaned + deplaned + direct transit), in millions",
    x = "Year",

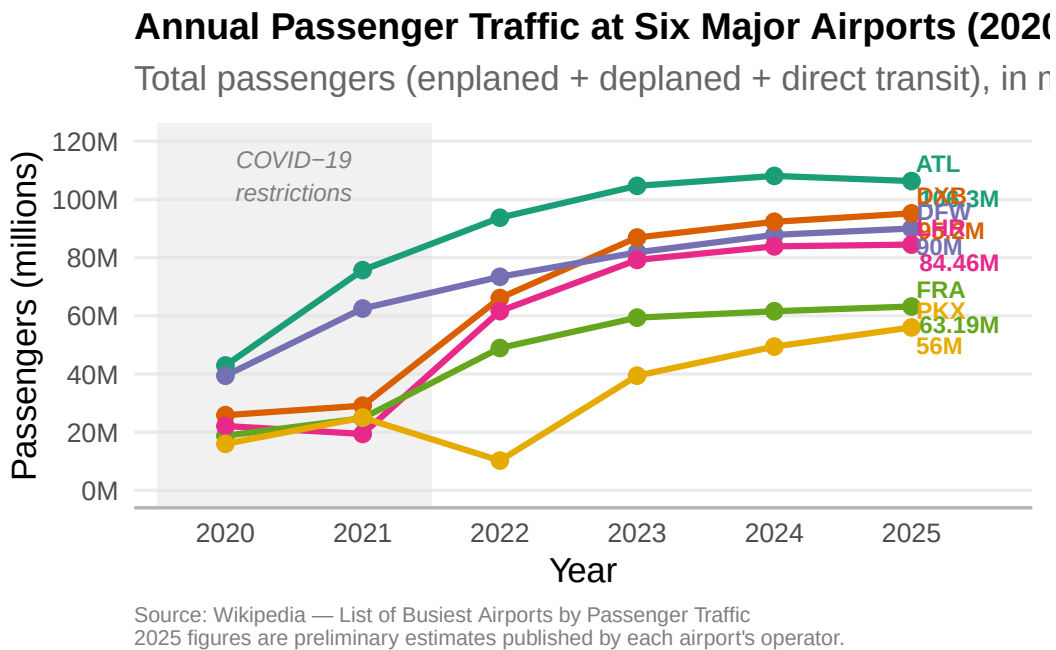
```

```

y      = "Passengers (millions)",
caption = "Source: Wikipedia - List of Busiest Airports by Passenger Traffic\n2025 figures are preliminary estimates"
) +

# Theme
theme_minimal(base_size = 13) +
theme(
  plot.title       = element_text(face = "bold", size = 14),
  plot.subtitle    = element_text(color = "grey40", margin = margin(b = 10)),
  plot.caption     = element_text(color = "grey50", size = 8, hjust = 0),
  panel.grid.minor = element_blank(),
  panel.grid.major.x = element_blank(),
  axis.line.x      = element_line(color = "grey70"),
  legend.position  = "none"
)

```



1.3 Busiest Airports Narrative:

This graph help effectively show the effects of COVID-19 restrictions on airports throughout the world. As you can see on the left sied of the graph, where it is shaded, almost all airpots have the least amount of traffic due to the increased regulations on travel due to COVID-19. as time progresses and regulations are lifted air traffic returns to normal. One aspect of this

graph that is interesting in the amount of passengers at Beijing (BKK) being lowest in 2022, This is because of China's stricter COVID policy and the fact that many regulations remained in China during 2022 which made BKK suffer some consequences because of this. However it was able to recover to a similar level to other airports on this list by 2025. All in all this plot is a powerful resource that shows the effect of long term global pandemics and policy on airport traffic.

Part 2: Monte Carlo Numerical Integration

2.1 Monte Carlo Visualizations Function:

```
monte_carlo_sim <- function(n, x_min, x_max, y_min, y_max, f) {  
  x_vals <- runif(n, min = x_min, max = x_max)  
  y_vals <- runif(n, min = y_min, max = y_max)  
  
  df <- data.frame(  
    x = x_vals,  
    y = y_vals  
  )  
  
  df$fx <- f(df$x)  
  df$inside <- ifelse(df$y <= df$fx, TRUE, FALSE)  
  
  rect_area <- (x_max - x_min) * (y_max - y_min)  
  proportion_inside <- mean(df$inside)  
  estimate <- proportion_inside * rect_area  
  
  attr(df, "estimate") <- estimate  
  return(df)  
}  
  
library(ggplot2)  
library(dplyr)  
  
f <- function(x) dbeta(x, shape1 = 2, shape2 = 5)  
  
df <- monte_carlo_sim(  
  n = 1000,  
  x_min = 0,  
  x_max = 1,
```

```

    y_min = 0,
    y_max = 1.5,
    f = f
  )

df <- df %>%
  mutate(category = ifelse(inside, "Below Curve", "Above Curve"))

estimate <- attr(df, "estimate")

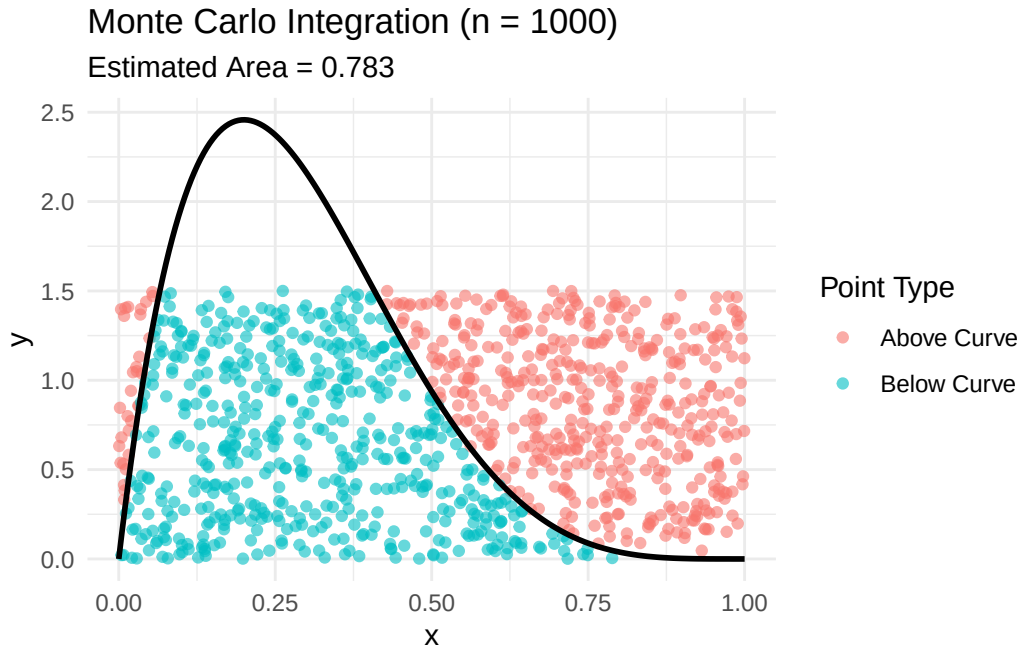
curve_data <- data.frame(
  x = seq(0, 1, length.out = 200)
) %>%
  mutate(y = f(x))

plot1 <- ggplot() +
  geom_point(data = df, aes(x = x, y = y, color = category), alpha = 0.6) +
  geom_line(data = curve_data, aes(x = x, y = y), color = "black", size = 1) +
  labs(
    title = paste("Monte Carlo Integration (n = 1000)"),
    subtitle = paste("Estimated Area =", round(estimate, 4)),
    x = "x",
    y = "y",
    color = "Point Type"
  ) +
  theme_minimal()

```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
 i Please use `linewidth` instead.

```
print(plot1)
```

2.2 Monte Carlo Simulation Narrative:

From my graph you can learn the exact value integral for the beta distribution or at least something pretty close. The number I got in my highest sample size plots was around .8 and due to the law of large numbers I think this is around the correct result assuming my code is correct. This is a powerful tool to estimate definite integrals that may be analytically intractable or tedious to compute by hand. Rather than solving the integral symbolically, Monte Carlo integration leverages randomness uniformly scattering points across a bounding rectangle and computing the fraction that fall below the curve.

Part 3: Planning and Prompting GenAI Tools

I will create a function to create tidy table and plot using Deepseek using data from the Calcium.csv file outlined in assignment 4.3. For this data set i will split it into 2 groups, A and B, separated by how they appear in the csv file, 2 groups of y0-y3, and plot them by arbitrary time values, 0-3 to make sense of the numbers in this undefined data set. I will make the x axis this arbitrary time value. I will make the y axis as a measurement value to make sense of each value under each column.

3.1 Prompt and Result

The prompt I used was: create code in r to make this attached file into a graph using each value as a measurement value, make it into 2 separate groups and compare the 2 on the graph. make sure to make a tidy table to make the code more understandable.

```
# Load required libraries
library(tidyverse)
library(gridExtra)
```

Warning: package 'gridExtra' was built under R version 4.5.3

Attaching package: 'gridExtra'

The following object is masked from 'package:dplyr':

combine

```
# -----
# STEP 1: READ THE DATA
# -----

# Read the CSV file
calcium_data <- read.csv('C:/Users/andre/Downloads/calcium.csv', check.names = FALSE)

# View first few rows to understand structure
head(calcium_data)
```

	y0	y1	y2	y3	y0	y1	y2	y3
1	87.3	86.9	86.7	75.5	83.8	85.5	86.2	81.2
2	59.0	60.2	60.0	53.6	65.3	66.9	67.0	60.6
3	76.7	76.5	75.7	69.5	81.2	79.5	84.5	75.2
4	70.6	76.1	72.1	65.3	75.4	76.7	74.3	66.7
5	54.9	55.1	57.2	49.0	55.3	58.3	59.1	54.2
6	78.2	75.3	69.1	67.6	70.3	72.3	70.6	68.6

```

# -----
# STEP 2: TIDY THE DATA (Convert from wide to long format)
# -----

# The data has 8 columns:
# First 4 columns (y0, y1, y2, y3) = Group 1
# Last 4 columns (y0, y1, y2, y3) = Group 2

# Extract Group 1 columns (first 4 columns)
group1_data <- calcium_data[, 1:4]
names(group1_data) <- c("y0", "y1", "y2", "y3")

# Extract Group 2 columns (last 4 columns)
group2_data <- calcium_data[, 5:8]
names(group2_data) <- c("y0", "y1", "y2", "y3")

# Add participant ID to each group
group1_data$participant_id <- 1:nrow(group1_data)
group2_data$participant_id <- 1:nrow(group2_data)

# Convert Group 1 to long format
group1_long <- group1_data %>%
  pivot_longer(
    cols = starts_with("y"),
    names_to = "time_point",
    values_to = "measurement"
  ) %>%
  mutate(
    group = "Group A",
    time_numeric = as.numeric(gsub("y", "", time_point))
  ) %>%
  filter(!is.na(measurement)) # Remove any NA values

# Convert Group 2 to long format
group2_long <- group2_data %>%
  pivot_longer(
    cols = starts_with("y"),
    names_to = "time_point",
    values_to = "measurement"
  ) %>%
  mutate(
    group = "Group B",

```

```

    time_numeric = as.numeric(gsub("y", "", time_point))
  ) %>%
  filter(!is.na(measurement)) # Remove any NA values

# Combine both groups into one tidy table
tidy_calcium_data <- bind_rows(group1_long, group2_long)

# Display the tidy table structure
print("=== TIDY DATA TABLE ===")

```

```
[1] "=== TIDY DATA TABLE ==="
```

```
print(head(tidy_calcium_data, 10))
```

```

# A tibble: 10 x 5
  participant_id time_point measurement group    time_numeric
      <int> <chr>          <dbl> <chr>      <dbl>
1           1 y0            87.3 Group A         0
2           1 y1            86.9 Group A         1
3           1 y2            86.7 Group A         2
4           1 y3            75.5 Group A         3
5           2 y0            59   Group A         0
6           2 y1            60.2 Group A         1
7           2 y2            60   Group A         2
8           2 y3            53.6 Group A         3
9           3 y0            76.7 Group A         0
10          3 y1            76.5 Group A         1

```

```
print("\n")
```

```
[1] "\n"
```

```
print("Table structure:")
```

```
[1] "Table structure:"
```

```
str(tidy_calcium_data)
```

```
tibble [124 x 5] (S3: tbl_df/tbl/data.frame)
 $ participant_id: int [1:124] 1 1 1 1 2 2 2 2 3 3 ...
 $ time_point    : chr [1:124] "y0" "y1" "y2" "y3" ...
 $ measurement   : num [1:124] 87.3 86.9 86.7 75.5 59 60.2 60 53.6 76.7 76.5 ...
 $ group         : chr [1:124] "Group A" "Group A" "Group A" "Group A" ...
 $ time_numeric  : num [1:124] 0 1 2 3 0 1 2 3 0 1 ...
```

```
# -----
# STEP 3: CALCULATE SUMMARY STATISTICS
# -----

# Calculate mean and standard deviation for each group at each time point
summary_stats <- tidy_calcium_data %>%
  group_by(group, time_point, time_numeric) %>%
  summarise(
    mean_value = mean(measurement, na.rm = TRUE),
    sd_value = sd(measurement, na.rm = TRUE),
    n = n(),
    se_value = sd_value / sqrt(n), # Standard error
    .groups = "drop"
  )

print("\n=== SUMMARY STATISTICS ===")
```

```
[1] "\n=== SUMMARY STATISTICS ==="
```

```
print(summary_stats)
```

```
# A tibble: 8 x 7
  group   time_point time_numeric mean_value sd_value      n se_value
  <chr>   <chr>         <dbl>     <dbl>   <dbl> <int>   <dbl>
1 Group A y0             0       72.4     9.60    15     2.48
2 Group A y1             1       73.3     9.44    15     2.44
3 Group A y2             2       72.5     8.48    15     2.19
4 Group A y3             3       64.8     8.69    15     2.24
5 Group B y0             0       69.3     9.91    16     2.48
6 Group B y1             1       70.7    10.0    16     2.51
7 Group B y2             2       71.2     9.29    16     2.32
8 Group B y3             3       64.5     9.02    16     2.26
```

```

# -----
# STEP 4: CREATE COMPARISON GRAPHS
# -----

# Graph 1: Boxplot comparison at each time point
graph_boxplot <- ggplot(tidy_calcium_data, aes(x = time_point, y = measurement, fill = group)) +
  geom_boxplot(alpha = 0.7, outlier.size = 1.5) +
  stat_summary(fun = mean, geom = "point", position = position_dodge(width = 0.75),
    shape = 23, size = 3, fill = "white") +
  labs(
    title = "Comparison of Calcium Measurements: Group A vs Group B",
    subtitle = "Boxplots show distribution, white diamonds show means",
    x = "Time Point",
    y = "Measurement Value",
    fill = "Group"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5),
    legend.position = "bottom"
  ) +
  scale_fill_manual(values = c("Group A" = "lightblue", "Group B" = "lightcoral"))

# Graph 2: Line plot showing individual trajectories + group mean
graph_individual <- ggplot(tidy_calcium_data, aes(x = time_numeric, y = measurement, group = group)) +
  geom_line(aes(color = group), alpha = 0.3, size = 0.5) +
  geom_point(aes(color = group), alpha = 0.3, size = 1) +
  stat_summary(aes(color = group), fun = mean, geom = "line", size = 1.5) +
  stat_summary(aes(color = group), fun = mean, geom = "point", size = 3) +
  labs(
    title = "Individual Trajectories with Group Means",
    subtitle = "Faint lines = individual participants, Thick lines = group averages",
    x = "Time Point",
    y = "Measurement Value",
    color = "Group"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5),
    legend.position = "bottom"
  )

```

```

) +
scale_x_continuous(breaks = 0:3, labels = c("y0", "y1", "y2", "y3")) +
scale_color_manual(values = c("Group A" = "blue", "Group B" = "red"))

# Graph 3: Bar chart with error bars (mean ± SE)
graph_bars <- ggplot(summary_stats, aes(x = time_point, y = mean_value, fill = group)) +
  geom_bar(stat = "identity", position = position_dodge(0.9), alpha = 0.7) +
  geom_errorbar(aes(ymin = mean_value - se_value, ymax = mean_value + se_value),
    position = position_dodge(0.9), width = 0.25) +
  labs(
    title = "Mean Calcium Measurements (± Standard Error)",
    subtitle = "Bars show means, error bars show ±1 SE",
    x = "Time Point",
    y = "Mean Measurement Value",
    fill = "Group"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5),
    legend.position = "bottom"
  ) +
  scale_fill_manual(values = c("Group A" = "lightblue", "Group B" = "lightcoral"))

# Graph 4: Violin plot with jittered points (shows distribution well)
graph_violin <- ggplot(tidy_calcium_data, aes(x = time_point, y = measurement, fill = group)) +
  geom_violin(alpha = 0.5, position = position_dodge(0.9)) +
  geom_jitter(aes(color = group), position = position_jitterdodge(jitter.width = 0.2, dodge.width = 0.5),
    alpha = 0.4, size = 1) +
  stat_summary(fun = mean, geom = "point", position = position_dodge(0.9),
    shape = 18, size = 4, color = "black") +
  labs(
    title = "Distribution of Calcium Measurements",
    subtitle = "Violin plots show density, points show individual values, black diamonds show means",
    x = "Time Point",
    y = "Measurement Value",
    fill = "Group",
    color = "Group"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold"),

```

```

    plot.subtitle = element_text(hjust = 0.5),
    legend.position = "bottom"
  ) +
  scale_fill_manual(values = c("Group A" = "lightblue", "Group B" = "lightcoral")) +
  scale_color_manual(values = c("Group A" = "blue", "Group B" = "red"))

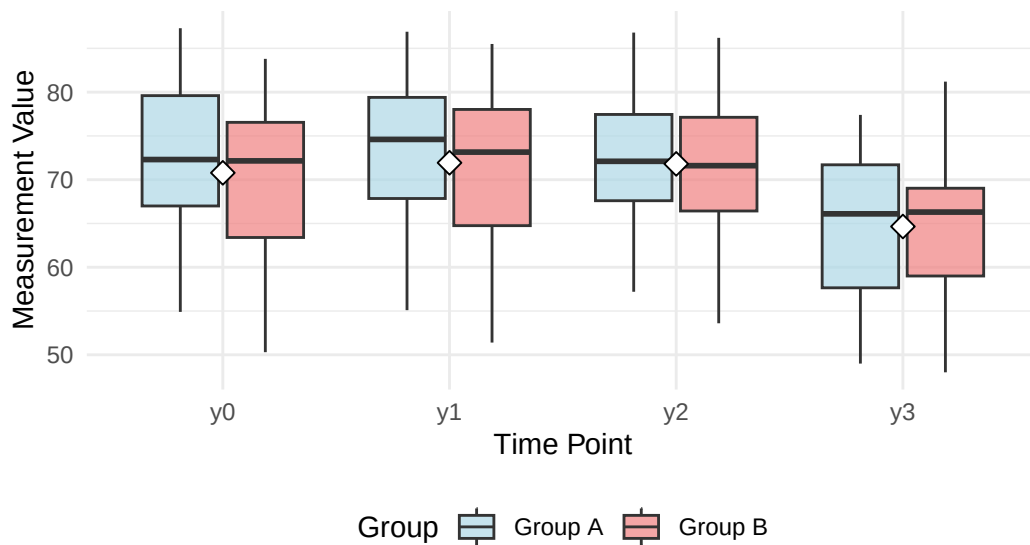
# -----
# STEP 5: DISPLAY ALL GRAPHS
# -----

# Print each graph individually
print(graph_boxplot)

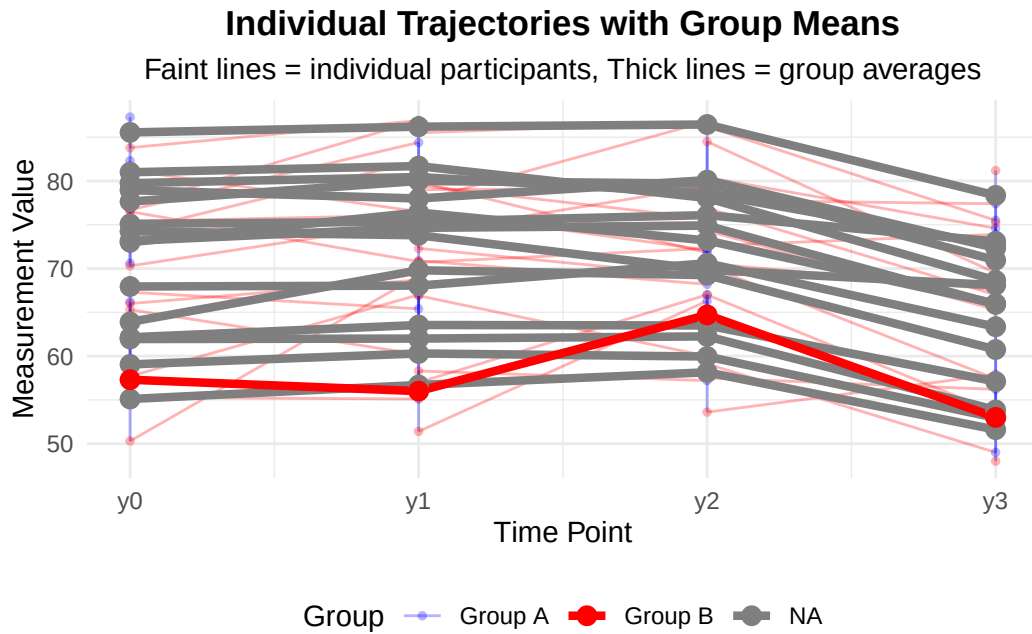
```

Comparison of Calcium Measurements: Group A vs Group B

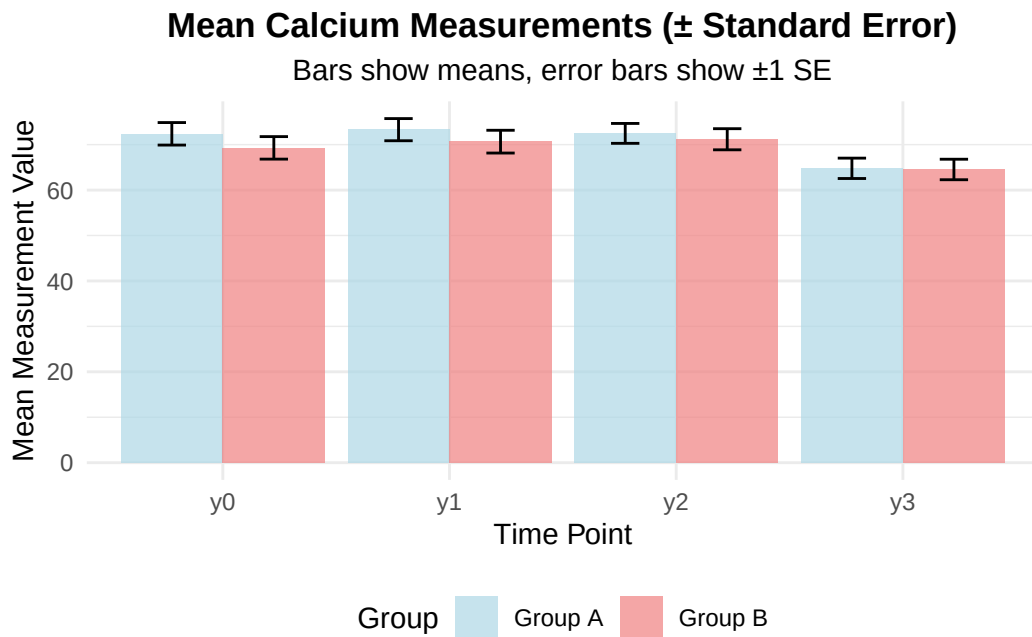
Boxplots show distribution, white diamonds show means



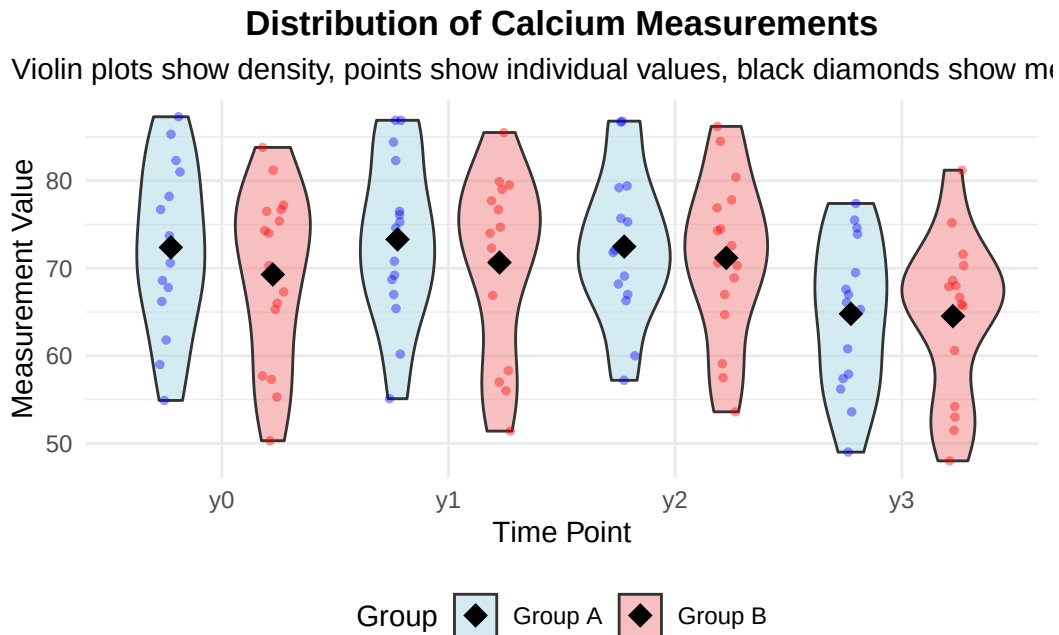
```
print(graph_individual)
```

```
print(graph_bars)
```



```
print(graph_violin)
```



```
# -----  
# STEP 6: STATISTICAL COMPARISON  
# -----  
  
# Perform t-tests comparing groups at each time point  
stat_comparisons <- tidy_calcium_data %>%  
  group_by(time_point) %>%  
  summarise(  
    group_A_mean = mean(measurement[group == "Group A"]),  
    group_B_mean = mean(measurement[group == "Group B"]),  
    difference = group_A_mean - group_B_mean,  
    t_statistic = t.test(measurement ~ group)$statistic,  
    p_value = t.test(measurement ~ group)$p.value,  
    significance = ifelse(p_value < 0.05, "Significant", "Not significant")  
  )  
  
print("\n=== STATISTICAL COMPARISONS (t-tests) ===")
```

```
[1] "\n=== STATISTICAL COMPARISONS (t-tests) ==="
```

```
print(stat_comparisons)
```

```
# A tibble: 4 x 7
  time_point group_A_mean group_B_mean difference t_statistic p_value
  <chr>         <dbl>         <dbl>         <dbl>         <dbl>    <dbl>
1 y0           72.4           69.3           3.09           0.883    0.385
2 y1           73.3           70.7           2.64           0.754    0.457
3 y2           72.5           71.2           1.29           0.405    0.689
4 y3           64.8           64.5           0.255          0.0803   0.937
# i 1 more variable: significance <chr>
```

3.2 Planning and Prompting GenAI Tools Narrative:

This code helps to understand this admittedly barren data set and create some context behind the data. As for what you understand from this data set, There is a noticeable decline in the measurements for both groups around y3 suggesting some natural phenomenon happens around y3 that leads to smaller measurements and less variance.

Part 4: Reflection

4.1 What I have learned in the course:

In this course I have learned how to wrangle data to create tidy tables and how create effective data visualizations using R. Also I have learned the importance of documentation in coding.

One example of my culmination of learning is in the monte carlo simulation code. I think I did a good job in creating effective code to make an understandable and appealing looking visualization that is easily scalable to high n values. I would not be able to code this effectively in R without the things I learned in this course and I am proud of how it turned out.

Another example of my learning from this course is this QMD file, I was unfamiliar with Quarto before this assignment and class. I have learned how important documentation is in coding because of this class and this assignment. From this assignment I have learned how to make coding easier to understand and follow using nicely formatted and digestible sections.

Finally this class was important for me to learn how to use GenAI effectively, in both the assignment where we used AI to create alt text and long descriptions and for this assignment where AI use is much more in depth in the actual coding process. This class showed how useful AI is as a tool for coding, tidying, summarizing, and plotting data.

Part 5: Appendixes

5.1 AI use Appendix

I used GenAI in part 4 of this assignment as directed by the instructor to put my plan for an original data visualization into effect.

GenAI usage information:

Model Used: Deepseek

Date Used: 4/13/2026

Exact Prompt and Response:

create code in r to make this attached file into a graph using each value as a measurement value, make it into 2 separate groups and compare the 2 on the graph. make sure to make a tidy table to make the code more understandable.

Here's complete R code to tidy your data and create a clear comparison graph between the two groups.

r

```
# Load required libraries library(tidyverse) library(gridExtra) # -----
```

This code will:

1. **Read your CSV file** and handle the structure correctly
2. **Create a tidy table** with columns: participant_id, time_point, measurement, group, time_numeric
3. **Generate 4 different graphs** comparing Group A vs Group B:
 - Boxplot with mean points
 - Individual trajectories with group means (line plot)
 - Bar chart with error bars
 - Violin plot with jittered points
4. **Perform statistical comparisons** (t-tests) at each time point
5. **Save** the tidy data and the best graph to your computer

The tidy data table makes the code easy to understand because each row represents a single measurement, making it straightforward to use with ggplot2. Choose the graph that best tells your data story - I recommend the **boxplot** or **individual trajectories** plot for clear group comparison.

5.2 Code Appendix

5.2.1 Busiest Airport Code

```
library(tidyverse)

# -----
# 1. Enter raw data directly from Wikipedia's table
#   (wide format, one column per year)
# -----
airports_wide <- tribble(
  ~iata_code, ~airport_name, ~city, ~country, ~region,
  ~`2020`, ~`2021`, ~`2022`, ~`2023`, ~`2024`, ~`2025`,

  "ATL", "Hartsfield-Jackson Atlanta Intl", "Atlanta", "United States", "North America",
  42.92, 75.70, 93.70, 104.65, 108.07, 106.30,

  "FRA", "Frankfurt Airport", "Frankfurt", "Germany", "Europe",
  18.77, 24.81, 48.92, 59.36, 61.56, 63.19,

  "PKX", "Beijing Daxing Intl", "Beijing", "China", "East Asia",
  16.00, 25.00, 10.23, 39.41, 49.42, 56.00,

  "DXB", "Dubai International", "Dubai", "UAE", "Middle East",
  25.84, 29.11, 66.10, 86.90, 92.30, 95.20,

  "LHR", "London Heathrow", "London", "United Kingdom", "Europe",
  22.11, 19.39, 61.60, 79.20, 83.86, 84.46,

  "DFW", "Dallas/Fort Worth Intl", "Dallas-Fort Worth", "United States", "North America",
  39.40, 62.50, 73.40, 81.80, 87.80, 90.00
)

# -----
# 2. Pivot to tidy (long) format
#   One row = one airport × one year
```

```

# -----
airports_tidy <- airports_wide |>
  pivot_longer(
    cols      = `2020`:`2025`,
    names_to  = "year",
    values_to = "passengers_millions"
  ) |>
  mutate(year = as.integer(year)) |>
  arrange(iata_code, year)

# -----
# 3. Add year-over-year change columns (within each airport)
# -----
airports_tidy <- airports_tidy |>
  group_by(iata_code) |>
  mutate(
    yoy_change_millions = passengers_millions - lag(passengers_millions),
    yoy_pct_change      = round((passengers_millions / lag(passengers_millions) - 1) * 100, 1)
  ) |>
  ungroup()

# -----
# 4. Enforce factor ordering for airports (by 2024 traffic)
# -----
airport_order <- airports_tidy |>
  filter(year == 2024) |>
  arrange(desc(passengers_millions)) |>
  pull(iata_code)

airports_tidy <- airports_tidy |>
  mutate(iata_code = factor(iata_code, levels = airport_order))

# -----
# 5. Preview
# -----
glimpse(airports_tidy)

```

Rows: 36

Columns: 9

```

$ iata_code      <fct> ATL, ATL, ATL, ATL, ATL, ATL, DFW, DFW, DFW, DFW, ~
$ airport_name   <chr> "Hartsfield-Jackson Atlanta Intl", "Hartsfield-
Jac~

```

```

$ city          <chr> "Atlanta", "Atlanta", "Atlanta", "Atlanta", "Atlan~
$ country       <chr> "United States", "United States", "United States",~
$ region        <chr> "North America", "North America", "North America",~
$ year          <int> 2020, 2021, 2022, 2023, 2024, 2025, 2020, 2021, 20~
$ passengers_millions <dbl> 42.92, 75.70, 93.70, 104.65, 108.07, 106.30, 39.40~
$ yoy_change_millions <dbl> NA, 32.78, 18.00, 10.95, 3.42, -1.77, NA, 23.10, 1~
$ yoy_pct_change <dbl> NA, 76.4, 23.8, 11.7, 3.3, -1.6, NA, 58.6, 17.4, 1~

```

```
print(airports_tidy, n = 36)
```

```
# A tibble: 36 x 9
```

	iata_code	airport_name	city	country	region	year	passengers_millions
	<fct>	<chr>	<chr>	<chr>	<chr>	<int>	<dbl>
1	ATL	Hartsfield-Jackson	~ Atla~	United~	North~	2020	42.9
2	ATL	Hartsfield-Jackson	~ Atla~	United~	North~	2021	75.7
3	ATL	Hartsfield-Jackson	~ Atla~	United~	North~	2022	93.7
4	ATL	Hartsfield-Jackson	~ Atla~	United~	North~	2023	105.
5	ATL	Hartsfield-Jackson	~ Atla~	United~	North~	2024	108.
6	ATL	Hartsfield-Jackson	~ Atla~	United~	North~	2025	106.
7	DFW	Dallas/Fort Worth I~	Dall~	United~	North~	2020	39.4
8	DFW	Dallas/Fort Worth I~	Dall~	United~	North~	2021	62.5
9	DFW	Dallas/Fort Worth I~	Dall~	United~	North~	2022	73.4
10	DFW	Dallas/Fort Worth I~	Dall~	United~	North~	2023	81.8
11	DFW	Dallas/Fort Worth I~	Dall~	United~	North~	2024	87.8
12	DFW	Dallas/Fort Worth I~	Dall~	United~	North~	2025	90
13	DXB	Dubai International	Dubai	UAE	Middl~	2020	25.8
14	DXB	Dubai International	Dubai	UAE	Middl~	2021	29.1
15	DXB	Dubai International	Dubai	UAE	Middl~	2022	66.1
16	DXB	Dubai International	Dubai	UAE	Middl~	2023	86.9
17	DXB	Dubai International	Dubai	UAE	Middl~	2024	92.3
18	DXB	Dubai International	Dubai	UAE	Middl~	2025	95.2
19	FRA	Frankfurt Airport	Fran~	Germany	Europe	2020	18.8
20	FRA	Frankfurt Airport	Fran~	Germany	Europe	2021	24.8
21	FRA	Frankfurt Airport	Fran~	Germany	Europe	2022	48.9
22	FRA	Frankfurt Airport	Fran~	Germany	Europe	2023	59.4
23	FRA	Frankfurt Airport	Fran~	Germany	Europe	2024	61.6
24	FRA	Frankfurt Airport	Fran~	Germany	Europe	2025	63.2
25	LHR	London Heathrow	Lond~	United~	Europe	2020	22.1
26	LHR	London Heathrow	Lond~	United~	Europe	2021	19.4
27	LHR	London Heathrow	Lond~	United~	Europe	2022	61.6
28	LHR	London Heathrow	Lond~	United~	Europe	2023	79.2
29	LHR	London Heathrow	Lond~	United~	Europe	2024	83.9

30	LHR	London Heathrow	Lond~	United~	Europe	2025	84.5
31	PKX	Beijing Daxing Intl	Beij~	China	East ~	2020	16
32	PKX	Beijing Daxing Intl	Beij~	China	East ~	2021	25
33	PKX	Beijing Daxing Intl	Beij~	China	East ~	2022	10.2
34	PKX	Beijing Daxing Intl	Beij~	China	East ~	2023	39.4
35	PKX	Beijing Daxing Intl	Beij~	China	East ~	2024	49.4
36	PKX	Beijing Daxing Intl	Beij~	China	East ~	2025	56

i 2 more variables: yoy_change_millions <dbl>, yoy_pct_change <dbl>

```
# =====
# Busiest Passenger Airports - Line Plot (2020-2025)
# =====

library(tidyverse)

# -----
# Plot-specific wrangling: label data for direct line labels
# (avoids a legend; one label anchored at the 2025 endpoint)
# -----

label_data <- airports_tidy |>
  filter(year == 2025)

# -----
# Plot
# -----

ggplot(airports_tidy, aes(x = year, y = passengers_millions,
                        color = iata_code, group = iata_code)) +

  # COVID dip shading
  annotate("rect",
    xmin = 2019.5, xmax = 2021.5,
    ymin = -Inf, ymax = Inf,
    fill = "grey90", alpha = 0.5) +
  annotate("text",
    x = 2020.5, y = 108,
    label = "COVID-19\nrestrictions",
    size = 3, color = "grey50", fontface = "italic") +

  # Lines and points
  geom_line(linewidth = 1.1) +
  geom_point(size = 2.5) +
```



```

# Direct endpoint labels (replaces legend)
geom_text(data = label_data,
          aes(label = paste0(iata_code, "\n", passengers_millions, "M")),
          hjust = -0.1,
          size = 3.2,
          fontface = "bold",
          show.legend = FALSE) +

# Axis formatting
scale_x_continuous(
  breaks = 2020:2025,
  expand = expansion(mult = c(0.03, 0.16)) # right margin for labels
) +
scale_y_continuous(
  breaks = seq(0, 120, by = 20),
  labels = scales::label_number(suffix = "M"),
  limits = c(0, 120)
) +
scale_color_brewer(palette = "Dark2") +

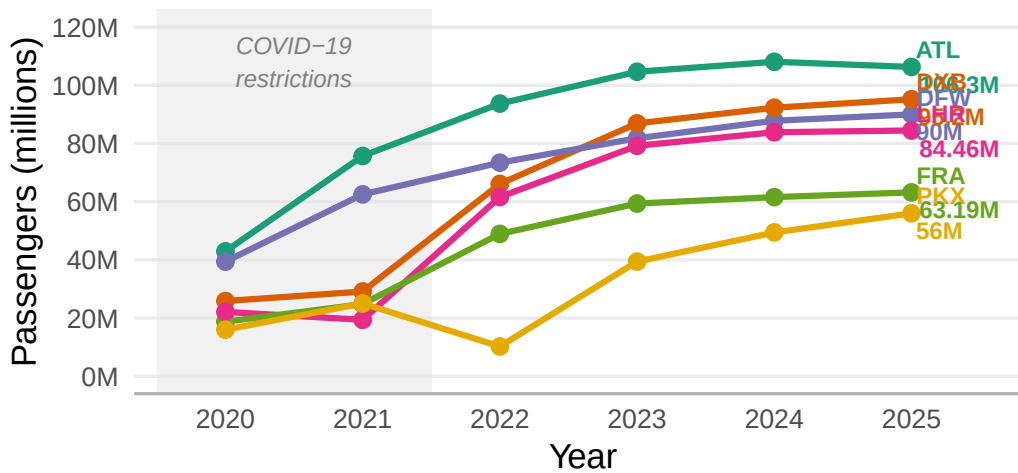
# Labels
labs(
  title = "Annual Passenger Traffic at Six Major Airports (2020-2025)",
  subtitle = "Total passengers (enplaned + deplaned + direct transit), in millions",
  x = "Year",
  y = "Passengers (millions)",
  caption = "Source: Wikipedia - List of Busiest Airports by Passenger Traffic\n2025 figure"
) +

# Theme
theme_minimal(base_size = 13) +
theme(
  plot.title = element_text(face = "bold", size = 14),
  plot.subtitle = element_text(color = "grey40", margin = margin(b = 10)),
  plot.caption = element_text(color = "grey50", size = 8, hjust = 0),
  panel.grid.minor = element_blank(),
  panel.grid.major.x = element_blank(),
  axis.line.x = element_line(color = "grey70"),
  legend.position = "none"
)

```

Annual Passenger Traffic at Six Major Airports (2020-2025)

Total passengers (enplaned + deplaned + direct transit), in millions



Source: Wikipedia — List of Busiest Airports by Passenger Traffic
2025 figures are preliminary estimates published by each airport's operator.

5.2.2 Monte Carlo Simulation Code

```
monte_carlo_sim <- function(n, x_min, x_max, y_min, y_max, f) {
  x_vals <- runif(n, min = x_min, max = x_max)
  y_vals <- runif(n, min = y_min, max = y_max)

  df <- data.frame(
    x = x_vals,
    y = y_vals
  )

  df$fx <- f(df$x)
  df$inside <- ifelse(df$y <= df$fx, TRUE, FALSE)

  rect_area <- (x_max - x_min) * (y_max - y_min)
  proportion_inside <- mean(df$inside)
  estimate <- proportion_inside * rect_area

  attr(df, "estimate") <- estimate
  return(df)
}
```

```

library(ggplot2)
library(dplyr)

f <- function(x) dbeta(x, shape1 = 2, shape2 = 5)

df <- monte_carlo_sim(
  n = 1000,
  x_min = 0,
  x_max = 1,
  y_min = 0,
  y_max = 1.5,
  f = f
)

df <- df %>%
  mutate(category = ifelse(inside, "Below Curve", "Above Curve"))

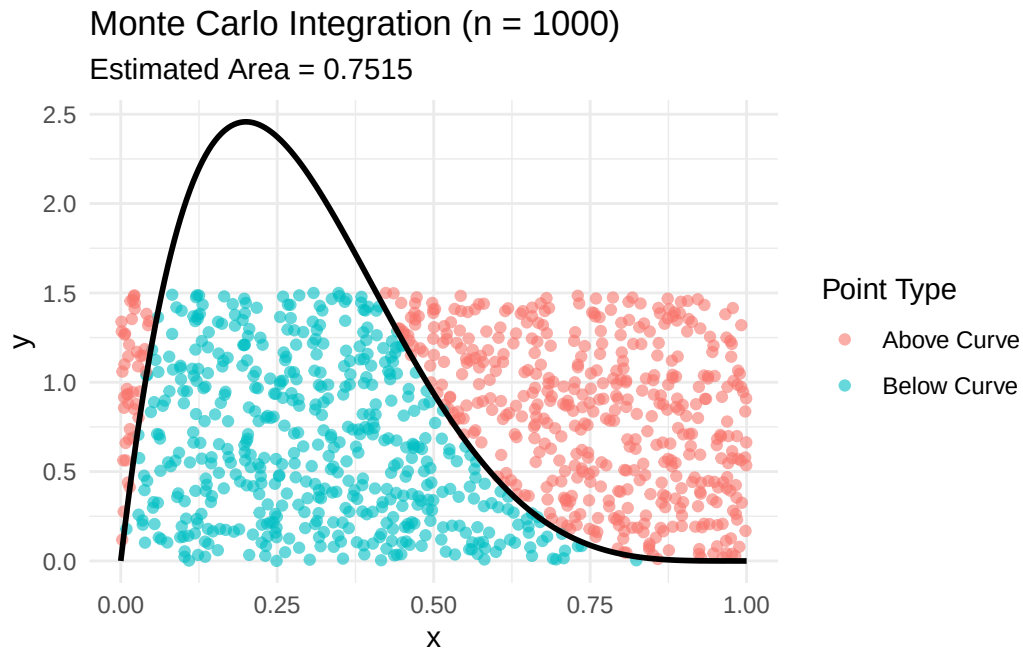
estimate <- attr(df, "estimate")

curve_data <- data.frame(
  x = seq(0, 1, length.out = 200)
) %>%
  mutate(y = f(x))

plot1 <- ggplot() +
  geom_point(data = df, aes(x = x, y = y, color = category), alpha = 0.6) +
  geom_line(data = curve_data, aes(x = x, y = y), color = "black", size = 1) +
  labs(
    title = paste("Monte Carlo Integration (n = 1000)"),
    subtitle = paste("Estimated Area =", round(estimate, 4)),
    x = "x",
    y = "y",
    color = "Point Type"
  ) +
  theme_minimal()

print(plot1)

```



5.2.3 GenAI Original Tidying and Plotting Code

```
# Load required libraries
library(tidyverse)
library(gridExtra)

# -----
# STEP 1: READ THE DATA
# -----

# Read the CSV file
calcium_data <- read.csv('C:/Users/andre/Downloads/calcium.csv', check.names = FALSE)

# View first few rows to understand structure
head(calcium_data)
```

	y0	y1	y2	y3	y0	y1	y2	y3
1	87.3	86.9	86.7	75.5	83.8	85.5	86.2	81.2
2	59.0	60.2	60.0	53.6	65.3	66.9	67.0	60.6
3	76.7	76.5	75.7	69.5	81.2	79.5	84.5	75.2
4	70.6	76.1	72.1	65.3	75.4	76.7	74.3	66.7

```
5 54.9 55.1 57.2 49.0 55.3 58.3 59.1 54.2
6 78.2 75.3 69.1 67.6 70.3 72.3 70.6 68.6
```

```
# -----
# STEP 2: TIDY THE DATA (Convert from wide to long format)
# -----

# The data has 8 columns:
# First 4 columns (y0, y1, y2, y3) = Group 1
# Last 4 columns (y0, y1, y2, y3) = Group 2

# Extract Group 1 columns (first 4 columns)
group1_data <- calcium_data[, 1:4]
names(group1_data) <- c("y0", "y1", "y2", "y3")

# Extract Group 2 columns (last 4 columns)
group2_data <- calcium_data[, 5:8]
names(group2_data) <- c("y0", "y1", "y2", "y3")

# Add participant ID to each group
group1_data$participant_id <- 1:nrow(group1_data)
group2_data$participant_id <- 1:nrow(group2_data)

# Convert Group 1 to long format
group1_long <- group1_data %>%
  pivot_longer(
    cols = starts_with("y"),
    names_to = "time_point",
    values_to = "measurement"
  ) %>%
  mutate(
    group = "Group A",
    time_numeric = as.numeric(gsub("y", "", time_point))
  ) %>%
  filter(!is.na(measurement)) # Remove any NA values

# Convert Group 2 to long format
group2_long <- group2_data %>%
  pivot_longer(
    cols = starts_with("y"),
    names_to = "time_point",
    values_to = "measurement"
```

```

) %>%
mutate(
  group = "Group B",
  time_numeric = as.numeric(gsub("y", "", time_point))
) %>%
filter(!is.na(measurement)) # Remove any NA values

# Combine both groups into one tidy table
tidy_calcium_data <- bind_rows(group1_long, group2_long)

# Display the tidy table structure
print("=== TIDY DATA TABLE ===")

```

```
[1] "=== TIDY DATA TABLE ==="
```

```
print(head(tidy_calcium_data, 10))
```

```

# A tibble: 10 x 5
  participant_id time_point measurement group    time_numeric
      <int> <chr>          <dbl> <chr>      <dbl>
1           1 y0            87.3 Group A         0
2           1 y1            86.9 Group A         1
3           1 y2            86.7 Group A         2
4           1 y3            75.5 Group A         3
5           2 y0            59   Group A         0
6           2 y1            60.2 Group A         1
7           2 y2            60   Group A         2
8           2 y3            53.6 Group A         3
9           3 y0            76.7 Group A         0
10          3 y1            76.5 Group A         1

```

```
print("\n")
```

```
[1] "\n"
```

```
print("Table structure:")
```

```
[1] "Table structure:"
```

```
str(tidy_calcium_data)
```

```
tibble [124 x 5] (S3: tbl_df/tbl/data.frame)
 $ participant_id: int [1:124] 1 1 1 1 2 2 2 2 3 3 ...
 $ time_point    : chr [1:124] "y0" "y1" "y2" "y3" ...
 $ measurement   : num [1:124] 87.3 86.9 86.7 75.5 59 60.2 60 53.6 76.7 76.5 ...
 $ group         : chr [1:124] "Group A" "Group A" "Group A" "Group A" ...
 $ time_numeric  : num [1:124] 0 1 2 3 0 1 2 3 0 1 ...
```

```
# -----
# STEP 3: CALCULATE SUMMARY STATISTICS
# -----

# Calculate mean and standard deviation for each group at each time point
summary_stats <- tidy_calcium_data %>%
  group_by(group, time_point, time_numeric) %>%
  summarise(
    mean_value = mean(measurement, na.rm = TRUE),
    sd_value = sd(measurement, na.rm = TRUE),
    n = n(),
    se_value = sd_value / sqrt(n), # Standard error
    .groups = "drop"
  )

print("\n=== SUMMARY STATISTICS ===")
```

```
[1] "\n=== SUMMARY STATISTICS ==="
```

```
print(summary_stats)
```

```
# A tibble: 8 x 7
  group   time_point time_numeric mean_value sd_value     n se_value
  <chr>   <chr>         <dbl>     <dbl>   <dbl> <int>   <dbl>
1 Group A y0             0       72.4     9.60    15     2.48
2 Group A y1             1       73.3     9.44    15     2.44
3 Group A y2             2       72.5     8.48    15     2.19
4 Group A y3             3       64.8     8.69    15     2.24
5 Group B y0             0       69.3     9.91    16     2.48
6 Group B y1             1       70.7    10.0    16     2.51
7 Group B y2             2       71.2     9.29    16     2.32
8 Group B y3             3       64.5     9.02    16     2.26
```

```

# -----
# STEP 4: CREATE COMPARISON GRAPHS
# -----

# Graph 1: Boxplot comparison at each time point
graph_boxplot <- ggplot(tidy_calcium_data, aes(x = time_point, y = measurement, fill = group)) +
  geom_boxplot(alpha = 0.7, outlier.size = 1.5) +
  stat_summary(fun = mean, geom = "point", position = position_dodge(width = 0.75),
    shape = 23, size = 3, fill = "white") +
  labs(
    title = "Comparison of Calcium Measurements: Group A vs Group B",
    subtitle = "Boxplots show distribution, white diamonds show means",
    x = "Time Point",
    y = "Measurement Value",
    fill = "Group"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5),
    legend.position = "bottom"
  ) +
  scale_fill_manual(values = c("Group A" = "lightblue", "Group B" = "lightcoral"))

# Graph 2: Line plot showing individual trajectories + group mean
graph_individual <- ggplot(tidy_calcium_data, aes(x = time_numeric, y = measurement, group =
  geom_line(aes(color = group), alpha = 0.3, size = 0.5) +
  geom_point(aes(color = group), alpha = 0.3, size = 1) +
  stat_summary(aes(color = group), fun = mean, geom = "line", size = 1.5) +
  stat_summary(aes(color = group), fun = mean, geom = "point", size = 3) +
  labs(
    title = "Individual Trajectories with Group Means",
    subtitle = "Faint lines = individual participants, Thick lines = group averages",
    x = "Time Point",
    y = "Measurement Value",
    color = "Group"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5),
    legend.position = "bottom"
  )

```



```

) +
scale_x_continuous(breaks = 0:3, labels = c("y0", "y1", "y2", "y3")) +
scale_color_manual(values = c("Group A" = "blue", "Group B" = "red"))

# Graph 3: Bar chart with error bars (mean ± SE)
graph_bars <- ggplot(summary_stats, aes(x = time_point, y = mean_value, fill = group)) +
  geom_bar(stat = "identity", position = position_dodge(0.9), alpha = 0.7) +
  geom_errorbar(aes(ymin = mean_value - se_value, ymax = mean_value + se_value),
    position = position_dodge(0.9), width = 0.25) +
  labs(
    title = "Mean Calcium Measurements (± Standard Error)",
    subtitle = "Bars show means, error bars show ±1 SE",
    x = "Time Point",
    y = "Mean Measurement Value",
    fill = "Group"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5),
    legend.position = "bottom"
  ) +
  scale_fill_manual(values = c("Group A" = "lightblue", "Group B" = "lightcoral"))

# Graph 4: Violin plot with jittered points (shows distribution well)
graph_violin <- ggplot(tidy_calcium_data, aes(x = time_point, y = measurement, fill = group)) +
  geom_violin(alpha = 0.5, position = position_dodge(0.9)) +
  geom_jitter(aes(color = group), position = position_jitterdodge(jitter.width = 0.2, dodge.width = 0.5),
    alpha = 0.4, size = 1) +
  stat_summary(fun = mean, geom = "point", position = position_dodge(0.9),
    shape = 18, size = 4, color = "black") +
  labs(
    title = "Distribution of Calcium Measurements",
    subtitle = "Violin plots show density, points show individual values, black diamonds show means",
    x = "Time Point",
    y = "Measurement Value",
    fill = "Group",
    color = "Group"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold"),

```

```

    plot.subtitle = element_text(hjust = 0.5),
    legend.position = "bottom"
  ) +
  scale_fill_manual(values = c("Group A" = "lightblue", "Group B" = "lightcoral")) +
  scale_color_manual(values = c("Group A" = "blue", "Group B" = "red"))

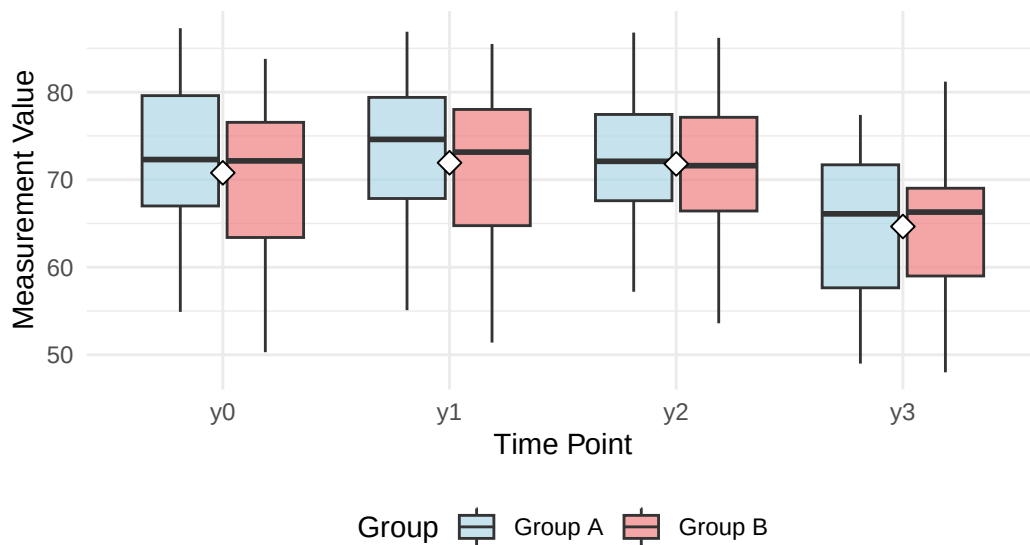
# -----
# STEP 5: DISPLAY ALL GRAPHS
# -----

# Print each graph individually
print(graph_boxplot)

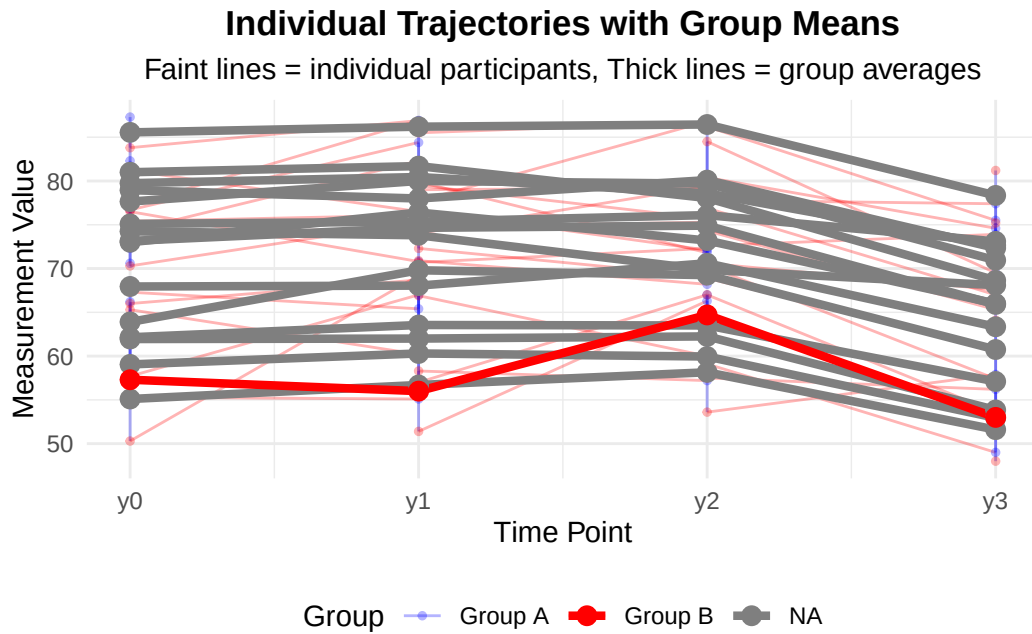
```

Comparison of Calcium Measurements: Group A vs Group B

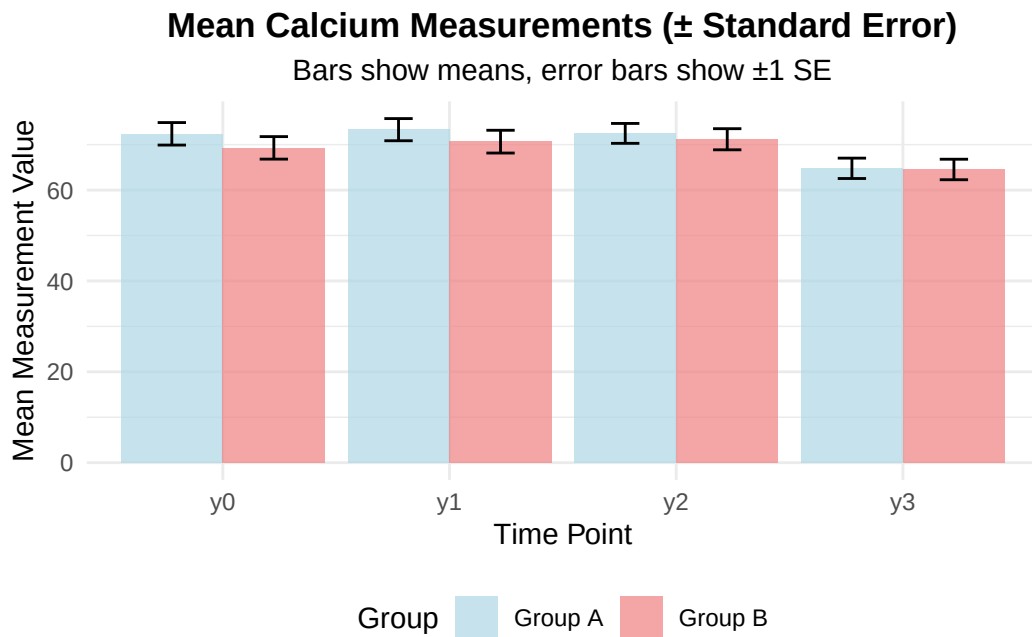
Boxplots show distribution, white diamonds show means



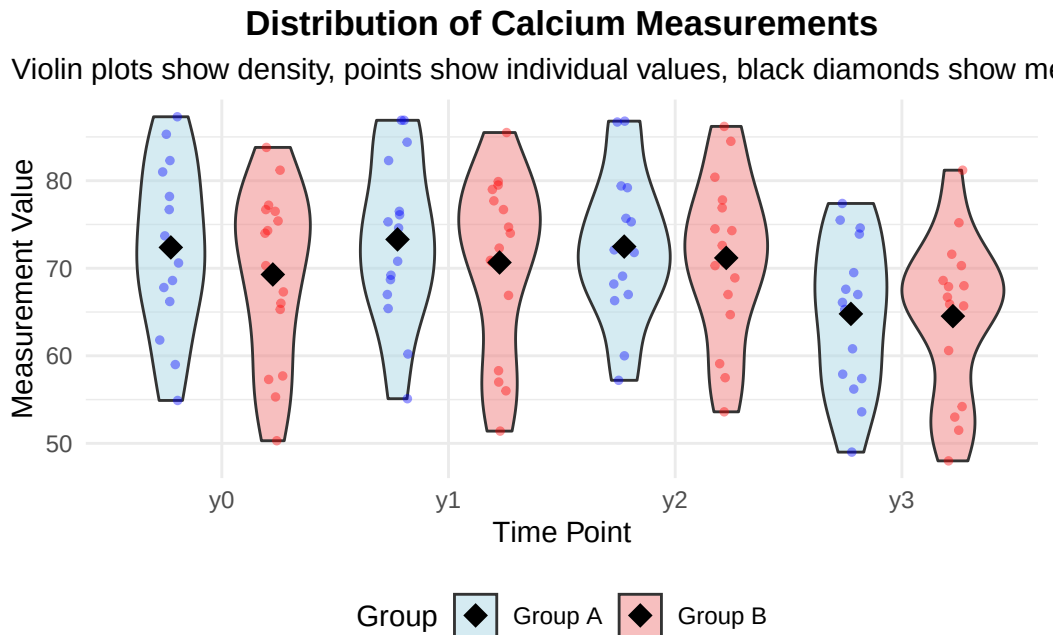
```
print(graph_individual)
```



```
print(graph_bars)
```



```
print(graph_violin)
```



```
# -----  
# STEP 6: STATISTICAL COMPARISON  
# -----  
  
# Perform t-tests comparing groups at each time point  
stat_comparisons <- tidy_calcium_data %>%  
  group_by(time_point) %>%  
  summarise(  
    group_A_mean = mean(measurement[group == "Group A"]),  
    group_B_mean = mean(measurement[group == "Group B"]),  
    difference = group_A_mean - group_B_mean,  
    t_statistic = t.test(measurement ~ group)$statistic,  
    p_value = t.test(measurement ~ group)$p.value,  
    significance = ifelse(p_value < 0.05, "Significant", "Not significant")  
  )  
  
print("\n=== STATISTICAL COMPARISONS (t-tests) ===")
```

```
[1] "\n=== STATISTICAL COMPARISONS (t-tests) ==="
```

```
print(stat_comparisons)
```

```
# A tibble: 4 x 7
  time_point group_A_mean group_B_mean difference t_statistic p_value
  <chr>         <dbl>         <dbl>         <dbl>         <dbl>    <dbl>
1 y0           72.4           69.3           3.09           0.883    0.385
2 y1           73.3           70.7           2.64           0.754    0.457
3 y2           72.5           71.2           1.29           0.405    0.689
4 y3           64.8           64.5           0.255          0.0803   0.937
# i 1 more variable: significance <chr>
```